

RECAP : Berechenbarkeit

- Da es überabzählbar viele Sprachen gibt, aber nur abzählbar viele TM, muss es nicht entscheidbare / nicht berechenbare Probleme geben.
- Ein praxisrelevantes nicht entscheidbares Problem ist das Halteproblem
- Um die Nichtentscheidbarkeit eines Problems zu beweisen, gehen wir von der Entscheidbarkeit aus und nutzen die TM, die dieses Problem entscheiden würde, um ein anderes bereits als nichtentscheidbar bekanntes Problem zu lösen :
 - Widerspruch durch Unterprogrammtechnik

RECAP

- Nicht entscheidbare Probleme

- Diagonalsprache: $D = \{w_i \mid M_i \text{ akzeptiert } w_i \text{ **nicht**}\}$
- Universalsprache: $U = \{\langle M \rangle w \mid M \text{ akzeptiert } w\}$
- Halteproblem: $H = \{\langle M \rangle w \mid M \text{ hält auf } w\}$
- Spezielles Halteproblem: $H_\epsilon = \{\langle M \rangle \mid M \text{ hält auf Eingabe } \epsilon\}$

- Ist L nicht entscheidbar, so ist auch das Komplement $\bar{L}$ nicht entscheidbar.

Berechenbarkeitstheorie

Satz von Rice

Da TM nicht auf jeder Eingabe halten, berechnen sie „partielle Funktionen“. Das können wir wie folgt formalisieren:

- Die von einer TM M berechnete Funktion ist von der Form

$$f_M: \{0,1\}^* \rightarrow \{0,1\}^* \cup \{\perp\}$$

Das Zeichen $\perp$ steht dabei für undefiniert und bedeutet, dass die Maschine nicht hält.
- Im Fall von Entscheidungsproblemen vereinfacht sich die Funktion zu

$$f_M: \{0,1\}^* \rightarrow \{0,1, \perp\}.$$

Dabei steht 0 für Verwerfen, 1 für Akzeptieren und $\perp$ für Nicht-Halten.

Satz von Rice

- Der Satz von Rice besagt, dass es unmöglich ist, eine beliebige nicht-triviale Eigenschaft eines Algorithmus bzw. einer TM algorithmisch zu entscheiden.
- Für spezielle Klassen von Algorithmen ist es zwar möglich Eigenschaften nachzuweisen, algorithmisch oder manuell. Es gibt aber keinen Algorithmus, der für **jeden** Algorithmus feststellen kann, ob dieser ein gewisses Verhalten zeigt.
- Spezialfälle: Halteproblem und Varianten, Schadcodeerkennung, Compileroptimierungen, Programmäquivalenz, Korrektheiterkennungen, ...

Satz von Rice

Satz: Sei R die Menge der von TM berechenbaren (partiellen) Funktionen und S eine Teilmenge von R mit $\emptyset \neq S \neq R$. Dann ist die Sprache $L(S) = \{\langle M \rangle \mid M \text{ berechnet eine Funktion aus } S\}$ nicht rekursiv.

Satz von Rice

Beweis:

- Wir nutzen die Unterprogrammtechnik. Aus einer TM $M_{L(S)}$, die $L(S)$ entscheidet, konstruieren wir eine TM M_ϵ , die das spezielle Halteproblem H_ϵ entscheidet. Vom speziellen Halteproblem H_ϵ wissen wir aber, dass es nicht entscheidbar ist.
- Wir beginnen mit einigen Definitionen und einer Annahme:
 - Sei u die überall undefinierte Funktion. OBdA $u \notin S$.
 - Sei $f \neq u$ eine Funktion aus S . Existiert, da $S \neq \emptyset$.
- Falls unsere obige Annahme über u nicht gilt, also im Falle $u \in S$, würden wir $f \in R - S \neq \emptyset$ wählen und das Akzeptanzverhalten der von uns im Folgenden konstruierten TM M_ϵ invertieren.

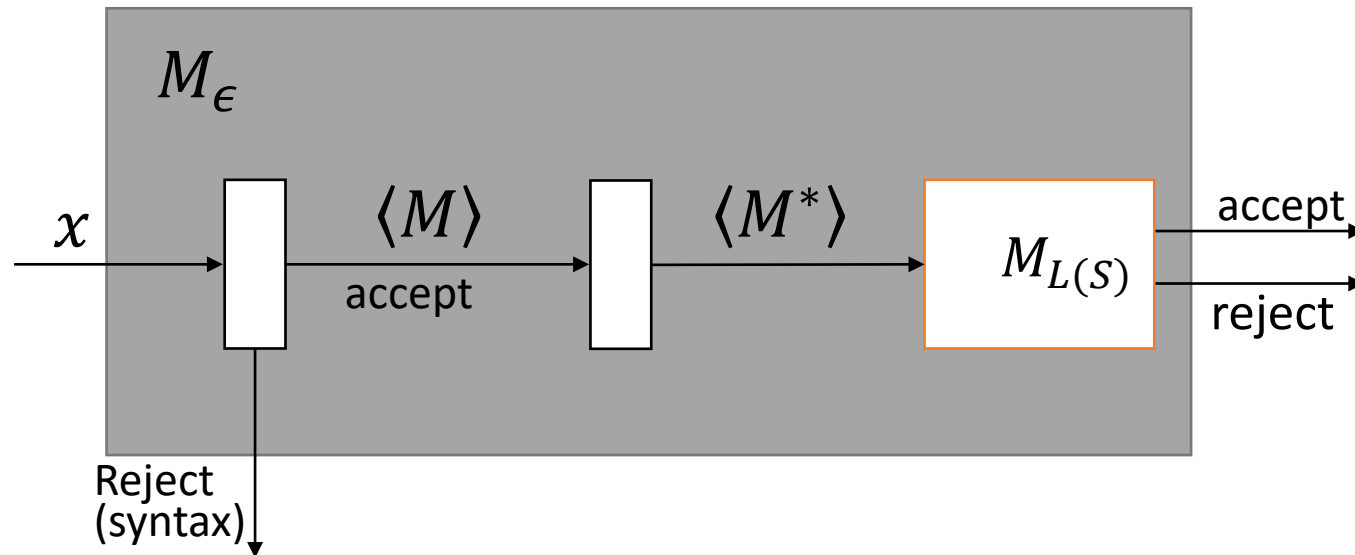
Satz von Rice

Beweis (cont.):

Die TM M_ϵ mit Unterprogramm $M_{L(S)}$ arbeitet wie folgt

- 1) Falls die Eingabe nicht aus einer korrekten Gödelnummer besteht, verwirft M_ϵ die Eingabe.
- 2) Sonst berechnet M_ϵ aus der Eingabe $\langle M \rangle$ die Gödelnummer einer TM M^* mit dem folgendem Verhalten auf Eingabe x :
 - Schritt A: Ignoriere die Eingabe x . Simuliere das Verhalten von M bei Eingabe ϵ auf einer für diesen Zweck reservierten Spur.
 - Schritt B: Berechne $f(x)$, d.h. simuliere M_f auf Eingabe x .
- 3) Nachdem M_ϵ die Gödelnummer $\langle M^* \rangle$ auf das Band geschrieben hat, startet sie $M_{L(S)}$ auf dieser Eingabe, und akzeptiert genau dann, wenn $M_{L(S)}$ akzeptiert.

Satz von Rice



Satz von Rice

Beweis (cont.):

Partielle Korrektheit: Bei Eingabe einer Gödelnummer $\langle M \rangle$ gilt:

$$\begin{aligned} M_\epsilon \text{ akzeptiert } \langle M \rangle &\Leftrightarrow M_{L(S)} \text{ akzeptiert } \langle M^* \rangle \\ &\Leftrightarrow \langle M^* \rangle \in L(S) \end{aligned}$$

$$\begin{aligned} (1) &\Leftrightarrow M^* \text{ erreicht Schritt } B \\ &\Leftrightarrow M \text{ hält auf Eingabe } \epsilon \\ &\Leftrightarrow \langle M \rangle \in H_\epsilon \end{aligned}$$

Die mit (1) markierte Äquivalenz begründen wir wie folgt:

- " $\Leftarrow$ ": Wenn M^* auf Eingabe x den Schritt B erreicht, so erreicht M^* diesen Schritt auf jeder Eingabe. Also ist M^* eine TM, die die Funktion f berechnet. Aus $f \in S$ folgt $\langle M^* \rangle \in L(S)$.
- " $\Rightarrow$ ": Wenn M^* Schritt B nicht erreicht, so wissen wir, dass M^* auf keiner Eingabe terminiert, also die Funktion $u \notin S$ berechnet. Also ist $\langle M^* \rangle$ nicht in $L(S)$.

Satz von Rice

Beweis (cont.):

Terminierung:

- Schritt 1 und 2 können in endlicher Zeit berechnet werden.
- In Schritt 3 wird $M_{L(S)}$ auf der Eingabe $\langle M^* \rangle$ gestartet. Nach unseren Annahmen hält $M_{L(S)}$ auf jeder Eingabe. Damit ist die Terminierung gesichert.

QED

Rekursiv aufzählbare Sprachen

- Eine Sprache L wird von einer TM M **erkannt**, wenn M jedes Wort aus L akzeptiert, und M kein Wort akzeptiert, das nicht in L enthalten ist.
- Eine Sprache L , für die eine TM existiert, die L erkennt, wird als **rekursiv aufzählbar** oder auch als **semi-entscheidbar** bezeichnet.
- Eine Sprache L wird von einer TM M **entschieden**, wenn M auf jeder Eingabe hält, und M genau die Wörter aus L akzeptiert.
- Eine Sprache L , für die eine TM existiert, die L entscheidet, wird als **rekursiv** oder auch als **entscheidbar** bezeichnet.

Rekursiv aufzählbare Sprachen

Bsp:

- Eine universelle TM erkennt die Universalsprache

$$U = \{\langle M \rangle w \mid M \text{ akzeptiert } w\}$$

- Damit ist die Universalsprache rekursiv aufzählbar. Die Universalsprache ist aber nicht rekursiv.

Rekursiv aufzählbare Sprachen

- Ein **Aufzähler** für eine Sprache $L \subseteq \Sigma^*$ ist eine Variante einer TM mit einem angeschlossenen Drucker im Sinne eines zusätzlichen Ausgabebandes, auf dem sich der Kopf nur nach rechts bewegt.
- Eigenschaften des Aufzählers
 - Gestartet mit leerem Arbeitsband, enummeriert der Aufzähler alle Wörter aus L auf dem Drucker, d.h.
 - gedruckt werden ausschließlich Wörter aus L , und
 - jedes Wort aus L wird irgendwann ausgedruckt.
- Die enummerierten Wörter sind dabei durch ein Zeichen getrennt, das nicht in Σ enthalten ist.
- Das wiederholte Ausdrucken von Wörtern aus L ist nicht explizit verboten.

Rekursiv aufzählbare Sprachen

Satz: Eine Sprache L ist genau dann rekursiv aufzählbar, wenn für L ein Aufzähler existiert.

Beweis:

Als erstes zeigen wir, wie man aus einem Aufzähler für L eine TM M konstruieren kann, die L erkennt.

- M simuliert den Aufzähler mit Hilfe einer Spur, die die Rolle des Druckers übernimmt.
- Wenn immer ein neues Wort enumeriert worden ist, vergleicht M dieses Wort mit w und akzeptiert bei Übereinstimmung.

Falls $w \in L$, so wird w irgendwann enumeriert und somit von M akzeptiert.

Falls $w \notin L$, so wird w nicht akzeptiert. Also erkennt M die Sprache L .

Rekursiv aufzählbare Sprachen

Beweis (cont.):

Zum Beweis der anderen Richtung zeigen wir nun, wie man aus einer TM M , die L erkennt, einen Aufzähler konstruiert.

Seien $w_1, w_2, \dots$ die Wörter aus Σ^* in kanonischer Reihenfolge.

Der Aufzähler arbeitet wie folgt.

- Für $i = 1, 2, 3, \dots$
 - Simuliere i Schritte von M auf jedem Wort $w_1, \dots, w_i$.
 - Wird dabei eines der Worte akzeptiert, so drucke es aus.

Falls das Wort w_k in L enthalten ist, so wird es von M nach einer endlichen Anzahl von Schritten, sagen wir nach t_k vielen Schritten, akzeptiert. In jeder Iteration $i \geq \max\{k, t_k\}$ druckt der Aufzähler dieses Wort somit aus.

Ein Wort $w \notin L$ wird hingegen nicht von M akzeptiert und somit auch nicht vom Aufzähler ausgedruckt.

Schnitte von Sprachen

Satz:

- Wenn die Sprachen L_1 und L_2 rekursiv sind, so ist auch die Sprache $L_1 \cap L_2$ rekursiv.
- Wenn die Sprachen L_1 und L_2 rekursiv aufzählbar sind, so ist auch die Sprache $L_1 \cap L_2$ rekursiv aufzählbar.

Sowohl die Menge der rekursiven als auch die Menge der rekursiv aufzählbaren Sprachen ist also gegen Schnittmengenbildung abgeschlossen.

Schnitte von Sprachen

Beweis:

Teil 1: Seien M_1 und M_2 zwei TM, die L_1 bzw. L_2 entscheiden. Wir konstruieren eine TM M , die $L_1 \cap L_2$ entscheidet.

- Auf Eingabe w simuliert M zunächst das Verhalten von M_1 auf w und dann das Verhalten von M_2 auf w .
- Falls M_1 und M_2 akzeptieren, so akzeptiert auch M .

M akzeptiert offensichtlich die Eingaben aus $L_1 \cap L_2$. Zudem hält M auf jeder Eingabe, da sowohl M_1 als auch M_2 auf jeder Eingabe halten. Also entscheidet M die Sprache $L_1 \cap L_2$.

Teil 2: Wir verwenden dieselbe Konstruktion wie in Teil 1. Alle Aussagen, bis auf die zur Terminierung, gelten wie zuvor, wenn wir jedes Vorkommen des Begriffes "entscheiden" durch den Begriff "erkennen" ersetzen.

QED

Vereinigung von Sprachen

Satz:

- Wenn die Sprachen L_1 und L_2 rekursiv sind, so ist auch die Sprache $L_1 \cup L_2$ rekursiv.
- Wenn die Sprachen L_1 und L_2 rekursiv aufzählbar sind, so ist auch die Sprache $L_1 \cup L_2$ rekursiv aufzählbar.

Sowohl die Menge der rekursiven als auch die Menge der rekursiv aufzählbaren Sprachen ist also gegen Vereinigung abgeschlossen.

Vereinigung von Sprachen

Beweis:

Teil 1: Seien M_1 und M_2 zwei TM, die L_1 bzw. L_2 entscheiden. Wir konstruieren eine TM M , die $L_1 \cup L_2$ entscheidet.

- Auf Eingabe w , simuliert M zunächst das Verhalten von M_1 auf w und dann das Verhalten von M_2 auf w .
- Falls M_1 oder M_2 akzeptieren, so akzeptiert auch M .

M akzeptiert offensichtlich die Eingaben aus $L_1 \cup L_2$. Zudem hält M auf jede Eingabe, da sowohl M_1 als auch M_2 auf jeder Eingabe halten. Also entscheidet M die Sprache $L_1 \cup L_2$.

Vereinigung von Sprachen

Beweis (cont.):

Teil 2:

Wir können diesmal die Konstruktion aus Teil 1 nicht ohne Weiteres auf Maschinen M_1 und M_2 übertragen, die die Sprachen L_1 und L_2 nur erkennen. Das Problem ist, dass möglicherweise M_2 die Eingabe akzeptiert, aber M das niemals feststellt, da M_1 nicht terminiert.

Statt der sequentiellen Simulation von M_1 und M_2 verwenden wir eine parallele Simulation der beiden TM:

- M verwendet jeweils ein Band für die Simulation von M_1 und M_2 .
- Die Zustandsmenge von M enthält das Kreuzprodukt $Q_1 \times Q_2$ der Zustandsmengen von M_1 und M_2
- M akzeptiert, sobald eine der beiden TM akzeptiert.

QED

Komplemente von Sprachen

Lemma: Sind sowohl $L \subseteq \Sigma^*$ als auch $\bar{L} = \Sigma^* \setminus L$ rekursiv aufzählbar, so ist L rekursiv.

Beweis: Seien M und $\bar{M}$ Maschinen, die L bzw. $\bar{L}$ erkennen.

Die TM M' entscheidet L durch eine parallele Simulation von M und $\bar{M}$ auf der Eingabe w :

- M' akzeptiert w , sobald M akzeptiert.
- M' verwirft w , sobald $\bar{M}$ akzeptiert.

Da entweder $w \in L$ oder $w \notin L$, tritt eines dieser Ereignisse nach endlicher Zeit ein, so dass die Terminierung von M' sichergestellt ist.

QED

Komplemente von Sprachen

Satz: Wenn die Sprache L rekursiv ist, so ist auch $\bar{L}$ rekursiv.

Die Menge der rekursiven Sprachen ist also abgeschlossen gegen Komplementbildung.

Beweis:

Betrachte eine beliebige rekursive Sprache L . Sei M_L eine TM, die L entscheidet. Wir erhalten eine TM $M_{\bar{L}}$, die $\bar{L}$ entscheidet, indem wir das Akzeptanzverhalten von M_L invertieren. Also ist $\bar{L}$ rekursiv.

QED

Komplemente von Sprachen

Im Gegensatz zur Menge der rekursiven Sprachen ist die Menge der rekursiv aufzählbaren Sprachen nicht gegen Komplementbildung abgeschlossen.

Beispiel: die Universalsprache U belegt die Aussage. U ist rekursiv aufzählbar, weil U von der universellen TM erkannt wird. Wäre $\overline{U}$ ebenfalls rekursiv aufzählbar, so wäre U nach dem Lemma rekursiv. Ein Widerspruch dazu, dass U nicht rekursiv ist. Also ist $\overline{U}$ nicht rekursiv aufzählbar.

Evaluation

- URL: [Link zur Evaluation](#)
- KENNWORT: KZSSL

